

Transfer Learning with LoRA

John Christensen johnchr@kth.se
August Filannino augustfi@kth.se
Samy Zougari zougari@kth.se
Lydia Nasser lhnasser@kth.se

May 24, 2026

Abstract

This project explores transfer learning techniques for image classification using the Oxford-IIIT Pet Dataset. A pre-trained ResNet34 model is fine-tuned to perform both binary (cat vs. dog) and multi-class breed classification for baseline evaluation. Additionally, Low-Rank Adaptation (LoRA) is evaluated to improve parameter-efficient fine-tuning performance.

1 Introduction

This project applies transfer learning to the Oxford-IIIT Pet dataset using a pre-trained ResNet34, covering binary cat/dog classification and 37-class breed classification. We compare adaptation strategies including fine-tuning different numbers of layers and gradual un-freezing, and study their behaviour under limited data and class imbalance. We then evaluate Low-Rank Adaptation(LoRA) on a pre-trained ResNet50, a parameter efficient method that freezes the backbone and trains only small low-rank adapters at each adapted layer. We explore the choice of rank, target convolutional layers, scaling factor α , and learning rate, and compare LoRA to full fine-tuning across training data sizes to test whether its smaller number of trainable parameters makes it more robust when labelled data is limited.

2 Related Work

LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning method introduced by Hu et al. (2021) in “LoRA: Low-Rank Adaptation of Large Language Models”. It works by freezing the pre-trained model weights and injecting trainable low-rank decomposition matrices into selected layers of the network. Instead of updating all parameters during fine-tuning, only these low-rank matrices are optimized, which significantly reduces the number of trainable parameters while maintaining performance comparable to full fine-tuning.

3 Data

The dataset used was the Oxford-IIIT Pet Dataset which is a benchmark dataset mainly used in computer vision for image classification, segmentation and object detection tasks. It consists of 37 categories of different pet breeds of dogs and cats with approximately 200 images per category. In total, the dataset consists of 7249 images which is split such that 3680 of them are for training and 3669 are for testing. We chose to use 80% of the training data for training and the remaining 20% for validation for model selection, i.e. 2944 for training and 736 for validation.

Each image is annotated with breed labels and species labels, i.e. cat or dog, and in addition to this the dataset provides pixel-level segmentation masks for purposes such as semantic segmentation. The different images exhibit variation in pose, scale, lighting conditions, and background complexity, which makes the dataset suitable for evaluating model robustness and generalization.

Originally released by the University of Oxford Visual Geometry Group, the dataset is widely used in deep learning research, particularly for transfer learning experiments using convolutional neural networks.

4 Methods

4.1 Binary Classification using Feature Extraction

For the first baseline evaluation, transfer learning using ResNet34 was experimented on by replacing the final classification layer to have 2 outputs for dog and cat. The final classification was then finetuned and evaluated using the data split described in data section such that the only weights that were changed were those of this final layer. The optimizers used were the Adam optimizer using a learning rate of 0.001 from python libraries.

4.2 Breed Classification

For the second baseline evaluation, transfer learning using ResNet34 was experimented on by replacing the final classification layer to have 37 outputs for all the different breeds of dog and cat. Similar to the binary classification task, the final classification layer was then finetuned using the datasplit and reusing the ResNet34’s model weights for all the previous layers. The same optimizers were used as in binary task.

4.3 Parameter-Efficient Fine-Tuning: LoRA

In this work, LoRA is used to fine-tune the ResNet50 in a parameter-efficient manner by keeping the pre-trained weights frozen and training only the injected low-rank adaptation modules. The rank and scaling factors are selected as hyperparameters, and LoRA layers are applied to the relevant linear components of the network. This setup allows adaptation to both the binary and multi-class classification tasks while significantly reducing the number of trainable parameters compared to full fine-tuning.

5 Experiments

5.1 Binary Classification

As seen in the figure 4, the test accuracy was as high as 0.9868 when classifying dog or cat using the ResNet34 model which may be due to the largely different features and latent variables of these two classes.

5.2 Breed Classification

As seen in the first figure 5, the same experiment was performed using the multi-class breed classification which performed slightly worse with a test accuracy of 0.8728 which may be due to there being less data available for each class and that there are some breeds that have very similar features and are therefore more difficult to distinguish. Although, these results are very promising since no advanced programming or state-of-the-art machine learning skills was necessary only transfer learning and replacing one layer.

5.2.1 Fine-tuning L layers simultaneously

As seen in the figure 6, the final test average accuracies over 5 runs achieved for each l is as following $l=1$ 0.8146, $l=2$ 0.7074, $l=3$ 0.6460 and for $l=4$ 0.6007. This may be unclear in the figure but each dotted axhline representing test accuracy for each configuration is in iterative order, meaning that the best configuration is $l=1$, second best is using $l=2$ and so on. The configuration that produces the best accuracy over the test set among these 4 configurations therefore is using $l=1$.

5.2.2 Gradual un-freezing

As seen in figure 7, gradual un-freezing un-freezes layers progressively during training rather than from the start. We trained the classification head (fc) for the first 5 epochs, then un-froze one residual block group every 5 epochs in the order **layer4**, **layer3**, **layer2**, **layer1**, for a total of 25 epochs across 5 stages.

We used discriminative learning rates: 10^{-3} for the classification head, 10^{-4} for **layer4** and **layer3**, and 10^{-5} for **layer2** and **layer1**. Earlier layers encode more general features and benefit from smaller updates. The optimizer was AdamW with weight decay 10^{-4} , rebuilt at each stage transition to include the newly un-frozen parameters.

Averaged over 5 runs with different seeds, the test accuracy was 0.8994 ± 0.0049 . The very low standard deviation indicates the training procedure is stable across seeds. This is higher than any of the simultaneous fine-tuning configurations in Section 5.2.1, the best of which was $l = 1$ at 0.8146. Two factors likely contribute: the discriminative learning rates protect the pre-trained features in earlier layers from large updates, and giving the classifier time to stabilize before deeper layers train provides a more meaningful gradient signal when those layers begin training. Some overfitting remains, however, as training accuracy approaches 1.0 while validation plateaus near 0.88–0.91. A caveat is that gradual un-freezing trains for 25 epochs while our Strategy 1 used 5 epochs per configuration, so the strategies are not given the same total compute.

5.2.3 Effect of data fractions and regularization

To study the role of labelled data, we re-ran the depth experiments on stratified subsets of the training set (5%, 10%, 100%), keeping the per-class proportions fixed. As shown in Figure 8, the linear probe ($l = 1$) is the strongest strategy at every fraction and its advantage widens sharply as data shrinks: it holds 0.656 at 5% while the deeper configurations collapse to around 0.32–0.35, whereas at 100% all depths land within a few points of each other (0.899 for $l = 1$ down to 0.814 for $l = 4$). Unfreezing more layers adds trainable parameters that cannot be constrained with few labelled images, so the extra capacity overfits rather than helps; this is exactly the low-data regime where the parameter efficiency of a small adaptation matters most.

We then tested whether augmentation (flips, small rotations, crops, mild scaling) and L_2 weight decay recover accuracy where data is scarce. Augmentation alone gave no benefit and slightly hurt at every fraction as shown in figure 9 (e.g. $0.716 \rightarrow 0.689$ at 5%), likely because a frozen backbone with only a linear head cannot exploit the extra input variability within the short training budget. L_2 was more useful (figure 10: its benefit was concentrated in the low-data regime, where a moderate decay ($\text{wd} = 10^{-2}$) lifted 5% validation accuracy from 0.716 to ≈ 0.749 , while at 100% the regularization made essentially no difference. This matches the expectation that stronger regularization helps most precisely when the training set is smallest and overfitting is the dominant failure mode. As these regularization sweeps were run on a single seed, the smaller gaps are within the range attributable to seed noise.

5.2.4 Imbalanced Classes

To study how the network behaves on a skewed training distribution, each of the 12 cat breeds was reduced to 20% of its original training size while the 25 dog classes were kept intact; the test split remained balanced. Throughout this section the configuration was held fixed: ResNet34 with only the final classification layer trained ($L = 1$), Adam at $\eta = 10^{-3}$, and an L_1 weight penalty on the trainable parameters. Four settings were compared: no compensation, weighted cross-entropy with class weights inversely proportional to training frequency (1), a class-balanced random sampler, and the two combined.

$$\mathcal{L}_{\text{WCE}} = - \sum_{i=1}^{37} w_i y_i \log p_i, \quad (1)$$

Without any compensation strategy the achieved test accuracy was **81,85%**. Applying weighted cross-entropy alone raised it to **82,34%**, while a class-balanced random sampler reached **82,58%**. Combining the two strategies gave the strongest result with **83,21%**. Results are reported in Table 1. Both compensation strategies recover roughly half a percentage point of test accuracy over the uncompensated baseline, and applying them jointly yields the strongest result with a gain of **1,4** points on test accuracy and **1,3** points on macro F_1 . Neither strategy closes the gap entirely, and we can only hypothesize that a handful of visually similar breed pairs continue to dominate most of the remaining misclassifications. The plots for validation accuracy, F1 scores and loss can be viewed in Figures 11, 12, and 13 respectively.

Table 1: Test and validation performance under imbalanced training (ResNet34, $L = 1$, L_1 regularisation).

Strategy	Test Acc	Test F_1	Val Acc	Val F_1
Baseline	0,8185	0,8165	0,8791	0,8784
Weighted CE	0,8234	0,8202	0,8832	0,8806
Oversampling	0,8258	0,8217	0,8750	0,8719
Weighted CE + Oversampling	0,8321	0,8297	0,8832	0,8784

5.3 LoRA Experiments

5.3.1 Compute and memory comparison

We compared the linear probe, LoRA (`conv1, conv2, conv3`, $r = 8$, $\alpha = 16$) and full fine-tuning on ResNet50 in terms of peak GPU memory and per-epoch time, using the same batch size and data loader. As seen in Figure 14, LoRA does not meaningfully reduce peak memory over full fine-tuning (≈ 2.98 GB against ≈ 3.21 GB), even though it trains fewer parameters. The memory saved by training fewer parameters is small compared to the memory needed to run the full backbone, which both methods still use. The linear probe is the only configuration with a small footprint (≈ 0.47 GB). The timing in Figure 15 agrees: LoRA is even slightly slower than full fine-tuning (≈ 52 – 58 s against ≈ 48 – 54 s), and the linear probe is fastest. The benefit of LoRA here is therefore not training memory or speed but the much smaller set of trained parameters.

5.3.2 Rank evaluation

We evaluated the LoRA rank $r \in \{8, 16, 32\}$ on ResNet50, using $\alpha = 2r$ and training for 15 epochs over 3 seeds per rank. The number of trainable parameters grows roughly linearly with the rank: 665,789 for $r = 8$, 1,255,765 for $r = 16$ and 2,435,717 for $r = 32$. The average test accuracy went in the opposite direction however: $r = 8$ gave 0.8873 ± 0.0072 , $r = 16$ gave 0.8403 ± 0.0137 and $r = 32$ gave 0.7593 ± 0.0120 . As seen in Figure 16, higher ranks not only ended up lower but also had a much noisier validation curve, while $r = 8$ stayed clearly above the others throughout training. This indicates that for this dataset the extra capacity given by larger ranks is not used to learn something useful and is more likely fitting noise, so $r = 8$ is the configuration we use in the rest of the experiments.

5.3.3 Lora target layer evaluation

We found that injecting LoRA into the `conv3` layers alone performed best, ahead of `conv1`, `conv2` or all three combined as shown in figure 18. This is a capacity effect rather than a compute one: adapting fewer convolutions adds fewer trainable parameters, which reduces overfitting on a small dataset like Oxford-IIIT Pet, consistent with the rank sweep where the smallest configuration also generalised best.

5.3.4 Lora Alpha evaluation

In Lora, alpha is a scaling hyperparameter which determines the strength of the low-rank update applied to the original model weights. The Lora update is scaled by a factor of alpha divided by rank r , which means that alpha directly controls how much influence the learned low-rank matrices have on the final weights. A larger alpha increases the effect of Lora contribution, while a smaller alpha reduces it which would make the model behave more similarly to not using Lora, i.e. the pretrained model.

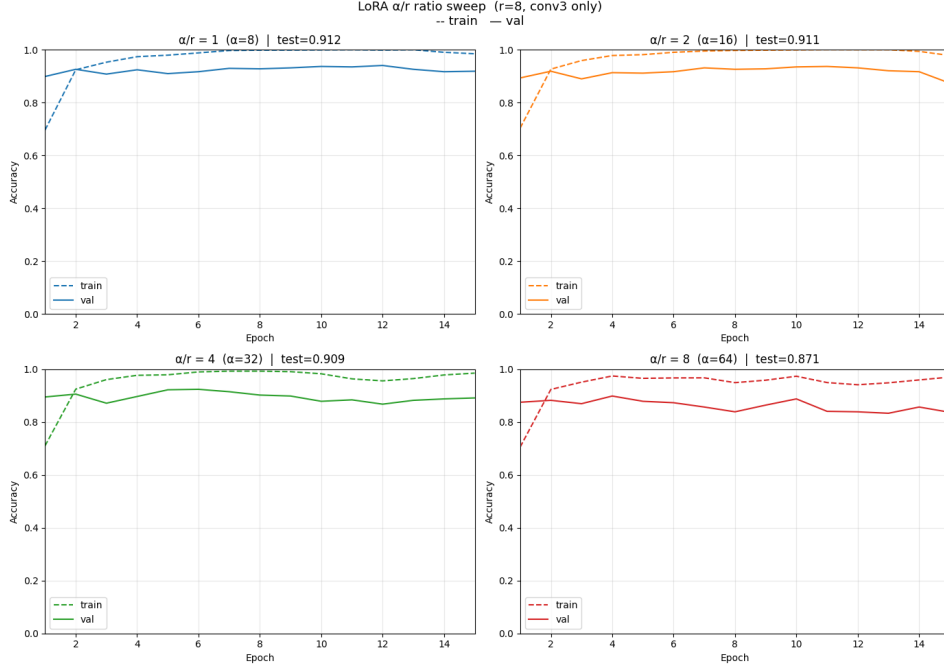


Figure 1: Comparison of accuracy for training, validation and test datasets over 1 run using lora for different configurations of the scaling factor alpha.

In Figure 1 above we can see that the effect on test accuracy does not make a big difference as previously thought, however the lower alpha i.e. the lower Lora contribution the better test accuracy by a very small margin but this may be due to noise in between the configurations using alpha as 8 or 16 but using a very high alpha of 64 reduces accuracy by almost 4 percent in comparison to using alpha of 8. There was not a significant difference in using 8, 16 or 32.

5.3.5 Lora learning rate evaluation

With the LoRA architecture determined from the previous tests, we tune the remaining optimization hyperparameter: the learning rate. Using alpha as 16 and rank as 8, we sweep the peak learning rate over a logspaced grid $\{1e-4, 3e-4, 1e-3, 3e-3\}$ and combine it with PyTorch’s built-in `CosineAnnealingLR` scheduler. This scheduler is used because it smoothly decays the learning rate from its peak to zero over training, which encourages large exploratory steps early on and finer adjustments near convergence (cosine slows down near the end). This matches the protocol used by the original LoRA paper [1]. Although methodology is matched, we use a different specific scheduler. The sweep peak LR and decay employment is using cosine instead of linear, with no warmup. We omit the warmup phase used in the paper since our training runs are short (15 epochs) and AdamW is reasonably robust at the LR scales we sweep. All other settings are held constant. As shown in Figure 17, we found that the max learning rate of $1e-3$ performed the best, with a test accuracy of **0.92**. Furthermore, configurations $1e-4$ and $3e-4$ had similar test accuracies of **0.918** and **0.919** respectively, and $3e-3$ performed considerable worse with test accuracy of **0.898**. These values inform further downstream analysis.

5.4 LoRA vs full fine-tuned ResNet on full and limited data evaluation

Based on the previous investigation on hyperparameters we set LoRA to $r = 8$, $\alpha = 16$, learning rate 10^{-3} with AdamW and 15 epochs. For full fine-tuning of ResNet50 we initially used the same learning rate of 10^{-3} since this

was the value that worked best in the earlier experiments, but this turned out to be too high when all parameters are trainable and the model ended up with very low test accuracy. We therefore lowered the learning rate to 10^{-4} for full fine-tuning. We then trained both setups on 100%, 50%, 10% and 5% of the training data and evaluated on the same test split. With the full training set the two methods land in the same range (LoRA ≈ 0.88 , full fine-tuning ≈ 0.87 test accuracy), but LoRA does this while updating only $\sim 0.67\text{M}$ parameters against the $\sim 23.6\text{M}$ of full fine-tuning, roughly $35\times$ fewer trainable parameters. So already on the full data LoRA matches full fine-tuning at a much lower parameter cost. As the training fraction goes down the gap between the two methods grows in favour of LoRA, as can be seen in Figure 2, and at 5% of the data LoRA reaches ≈ 0.76 while full fine-tuning drops to ≈ 0.67 . Figure 3 shows the training and validation curves for each fraction: at 100% both methods converge to similar validation accuracy while training accuracy reaches close to 1.0, but at 5% and 10% the full fine-tuning val curve plateaus noticeably lower than LoRA’s even though the train curve still goes to 1.0, which suggests that updating all parameters overfits more aggressively when the data is small.

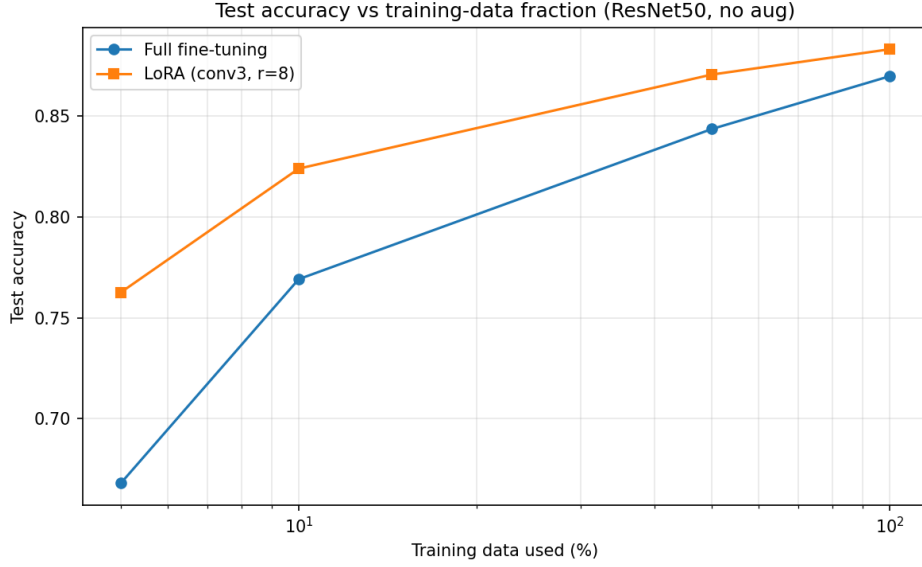


Figure 2: Test accuracy as a function of the training-data fraction (5%, 10%, 50%, 100%) for LoRA ($r = 8$, conv3) and full fine-tuning of ResNet50.

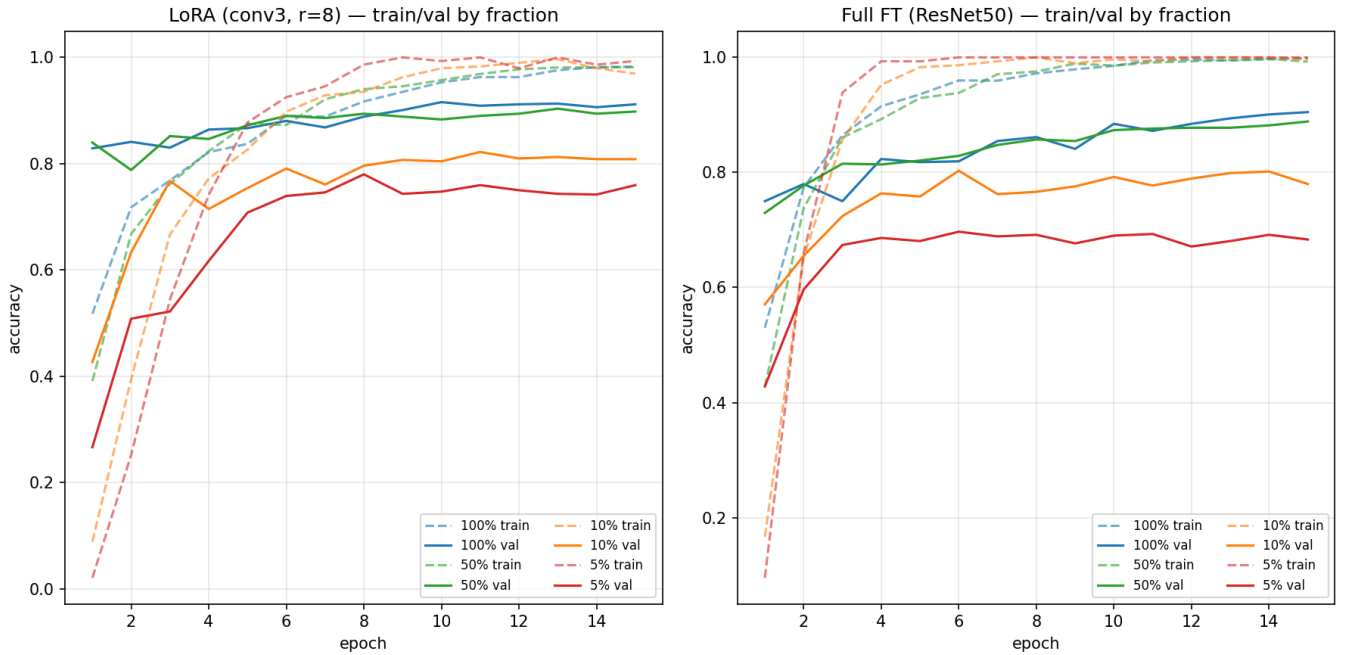


Figure 3: Train (dashed) and validation (solid) accuracy per epoch for LoRA (left) and full fine-tuning (right), shown for each training-data fraction.

6 Conclusion

As seen in Figure 6, comparing and evaluating test accuracy over different configurations of fine-tuning depth averaged over 5 runs using 5 epochs over the data shows that the best configuration is to only fine-tune $l=1$ i.e. as few layers as possible. In comparison to Figure 5, we can see that only fine-tuning the classification layer gives an even better test accuracy of 0.8728, averaged over 5 runs and 5 epochs. This is most likely due to the model being used for transfer learning, i.e. ResNet34, already being fine-tuned using better training schemes and methods. Therefore our results indicate that it is better to simply use the already fine-tuned model on more general data to then fine-tune only the classification layer. Using these results in comparison to Figure 4, it also shows that the model learns better representations given fewer classes to classify in the classification layer since the model achieved an even better test accuracy of 0.9868.

Gradual unfreezing improved on the L-layer results for the multi-class task, reaching a test accuracy of around 90% over 5 runs. Under limited data, the linear probe was the best configuration at every fraction tested, adding augmentation gave no improvement, while L_2 regularization helped slightly at the smallest data fractions. On the imbalanced version of the task, combining weighted cross entropy with class-balanced sampling gave the best result of 0.8321, recovering part of the accuracy lost to class imbalance.

For the Lora experiments, we found that applying Lora to the `conv3` layers of ResNet50 with rank $r = 8$ and $\alpha = 16$ achieved test accuracy comparable to full fine-tuning while training far fewer parameters. LoRA also held up better than full fine-tuning at smaller training fractions, supporting the hypothesis that its lower parameter count reduces overfitting when labelled data is limited. This makes LoRA a practical choice when training data or compute is constrained.

There are several natural ways to extend these experiments. Applying quantization (QLoRA) would let us pursue the memory savings that LoRA did not deliver here. It would also be worth investigating more closely why LoRA’s memory footprint stayed so close to full fine-tuning, and testing whether its savings become significant on a much larger backbone.

References

- [1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv preprint arXiv:2106.09685, 2021. <https://arxiv.org/pdf/2106.09685>

A Additional Results

A.1 Binary Classification

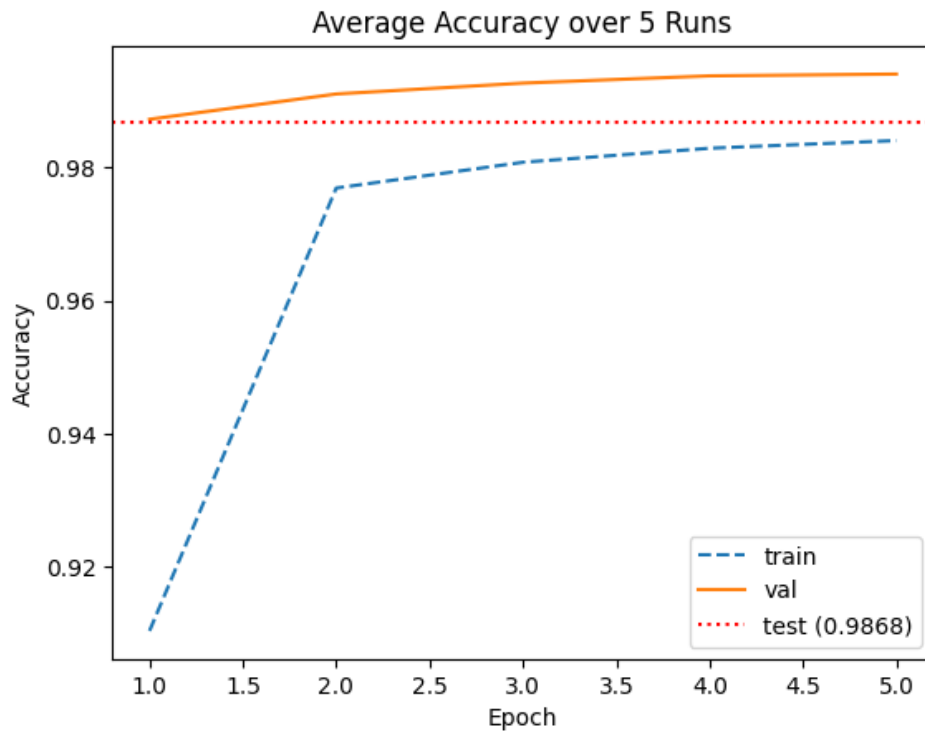


Figure 4: Binary classification over 5 runs with 5 epochs, learning rate = 0.001 using Adam. Figure displays test axhline accuracy and train and validation accuracy curves averaged.

A.2 Breed Classification

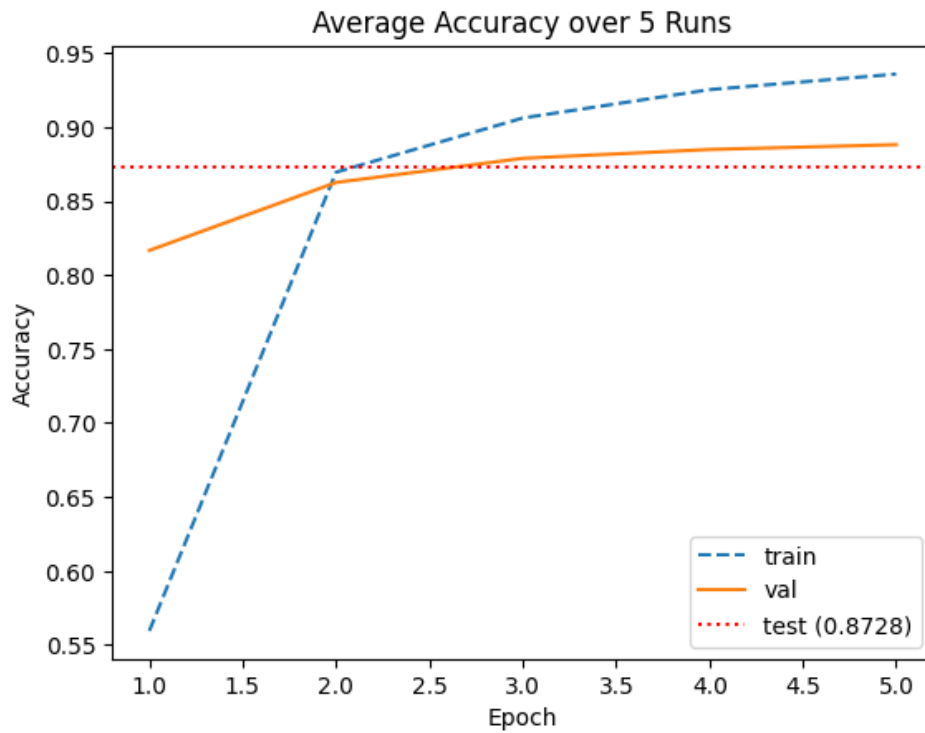


Figure 5: Multi-class breed classification over 5 runs with 5 epochs, learning rate = 0.001 using Adam. Figure displays test axhline accuracy and train and validation accuracy curves averaged.

A.3 Fine-tuning L layers simultaneously

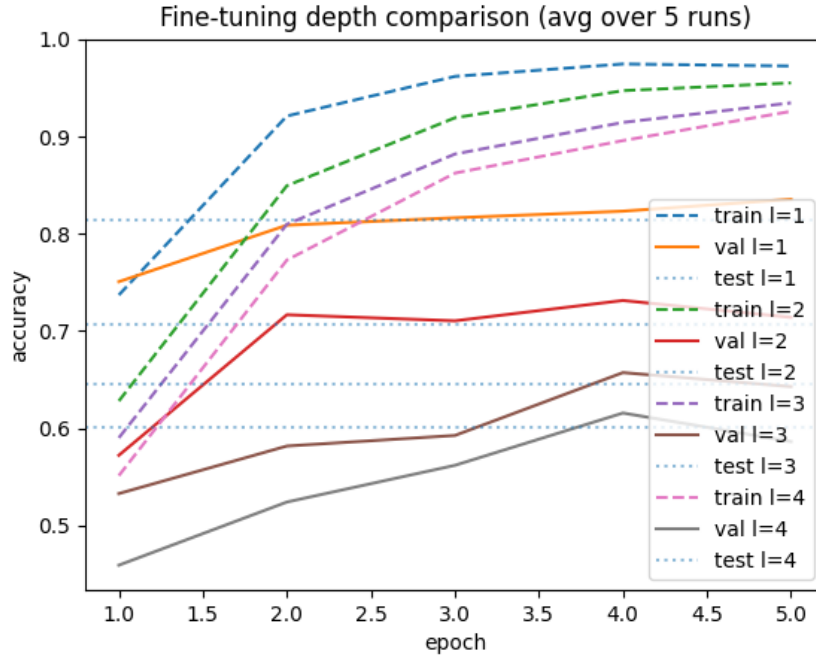


Figure 6: Multi-class breed classification over 5 runs with 5 epochs, learning rate = 0.001 using Adam. Figure displays test axhline accuracy and train and validation accuracy curves averaged. Comparing fine-tuning of different depths.

A.4 Gradual unfreezing

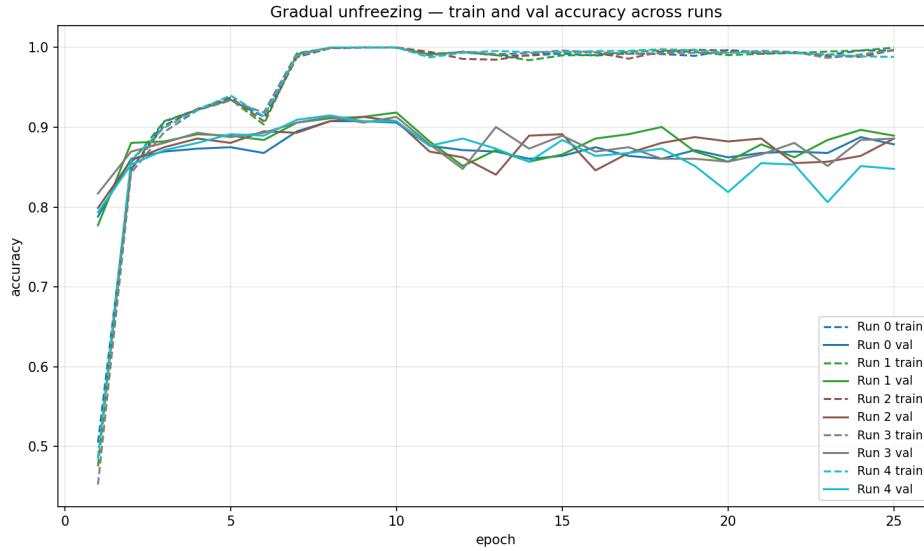


Figure 7: Gradual unfreezing over 5 runs, with each stage lasting 5 epochs, using AdamW with a learning rate of 10^{-3} for the classifier head.

A.5 Limited data

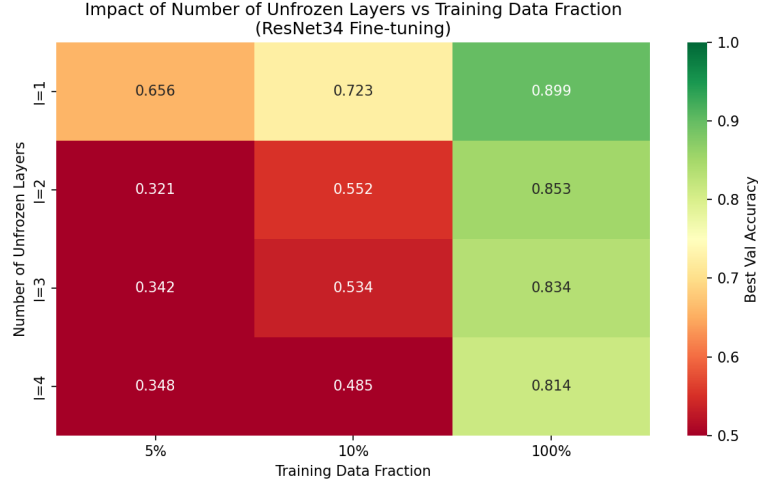


Figure 8: Best validation accuracy by number of unfrozen layers and training fraction (ResNet34). The linear probe ($l = 1$) is the best setup at every fraction.

A.6 Effect of data augmentation

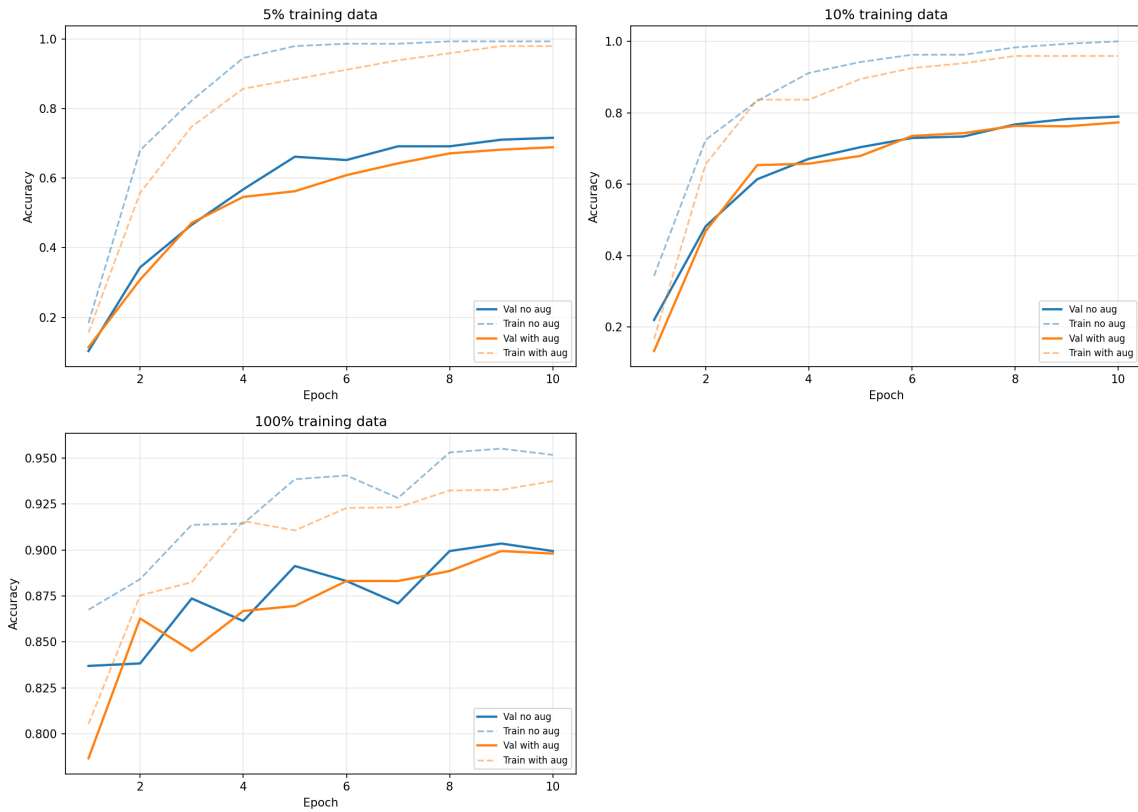


Figure 9: Effect of data augmentation (no L_2) on the linear probe across training fractions. Augmentation slightly lowers validation accuracy at every fraction.

A.7 Effect of L2 regularization

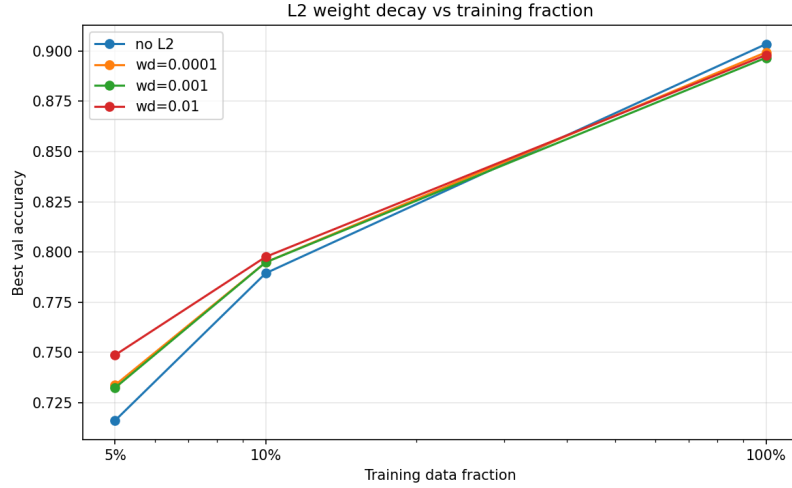


Figure 10: Effect of L_2 weight decay (no augmentation) on the linear probe across training fractions. The benefit is concentrated at 5% and vanishes at 100%.

A.8 Imbalanced classes

The following figures complement the imbalance experiment reported in Section 5.2.4.

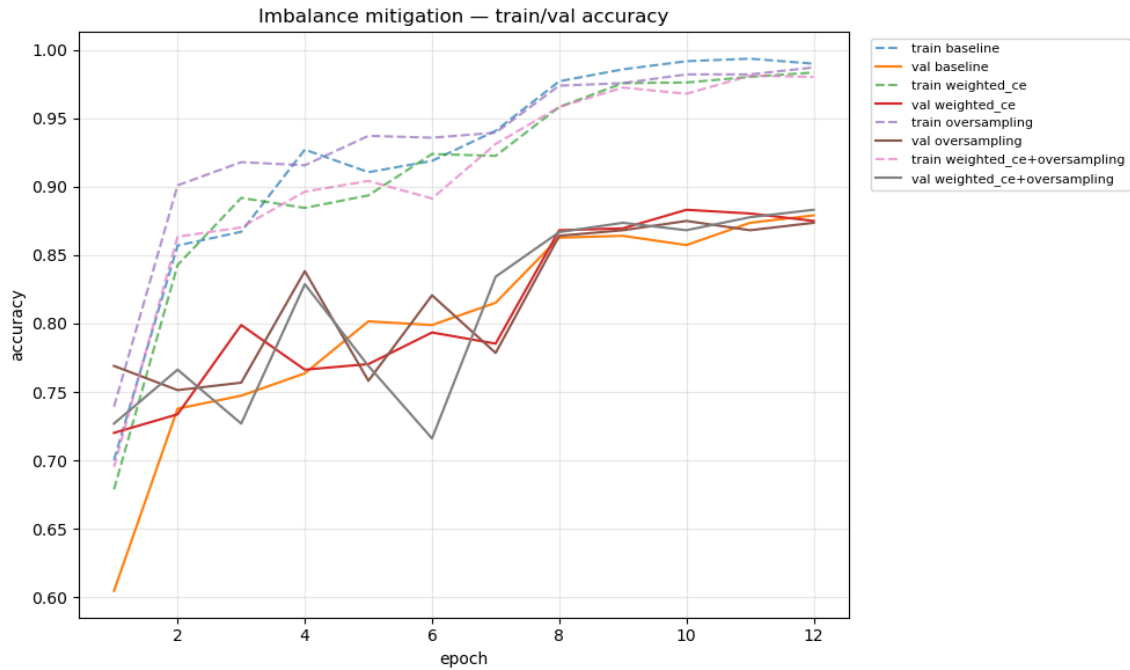


Figure 11: Validation accuracy per epoch for each compensation strategy. The combined weighted-CE + oversampling configuration converges to the highest validation accuracy.

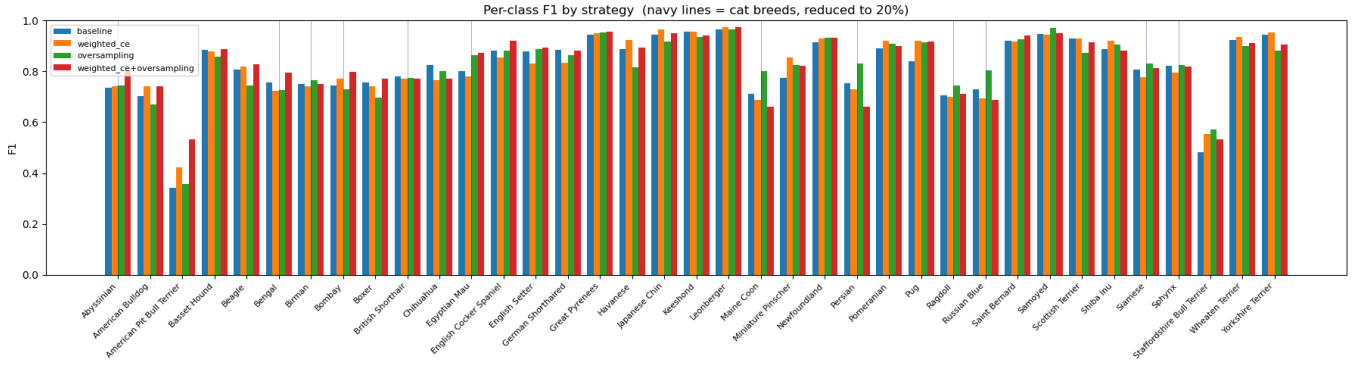


Figure 12: Per-class test F_1 for the four strategies. The combined strategy underperforms on e.g. Main Coon and Persian.

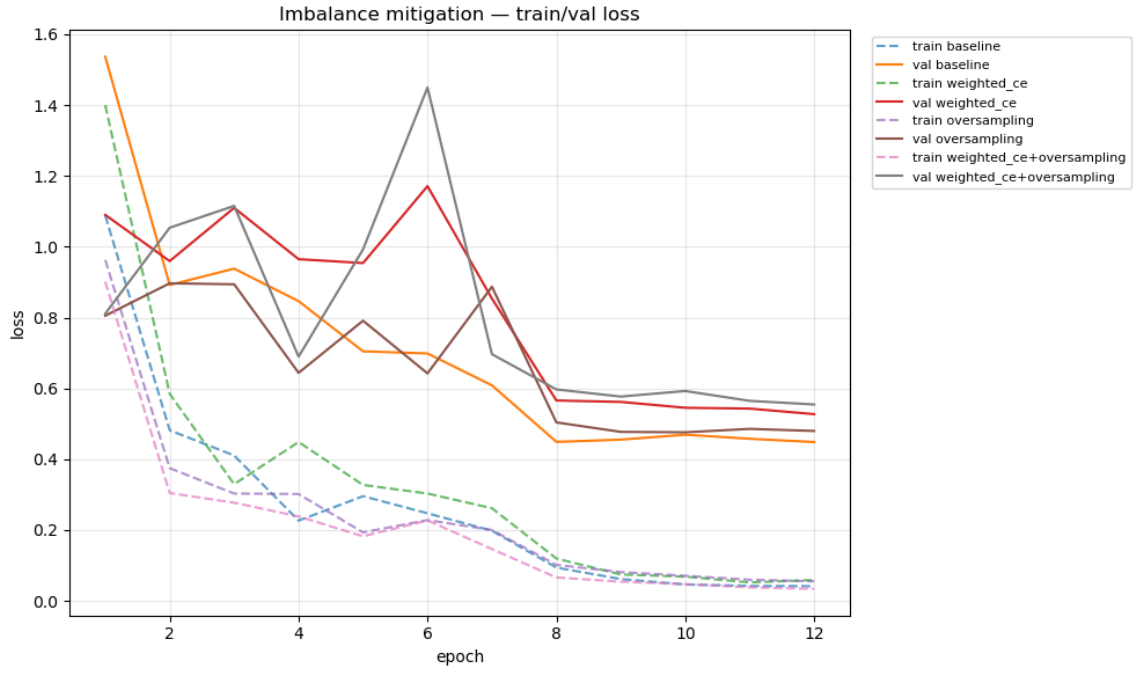


Figure 13: Training and validation loss curves

A.9 LoRA

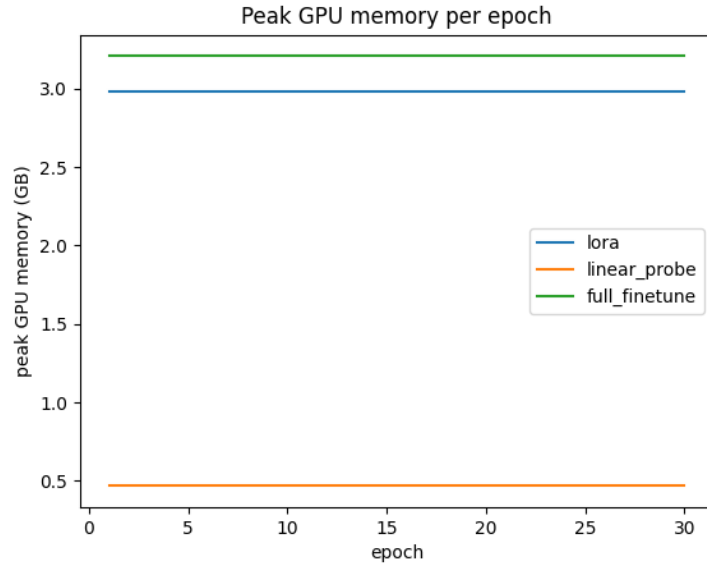


Figure 14: Peak GPU memory per epoch for linear probe, LoRA (`conv1`, `conv2`, `conv3`, $r = 8$) and full fine-tuning of ResNet50.

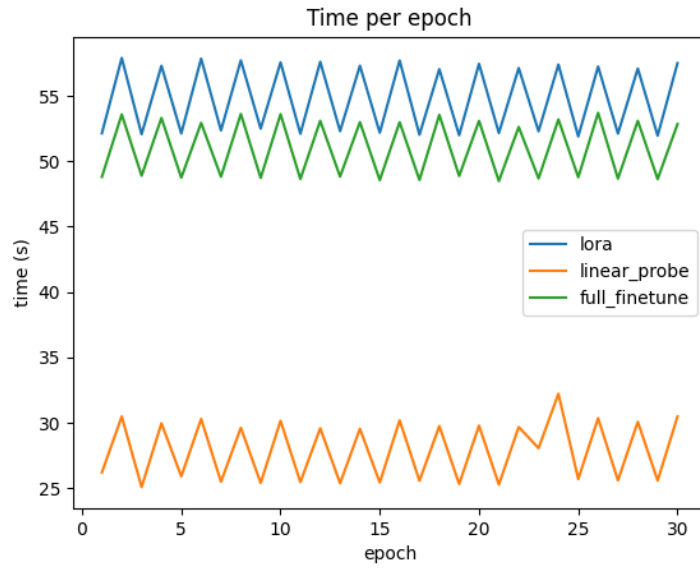


Figure 15: Wall-clock time per epoch for the three strategies.

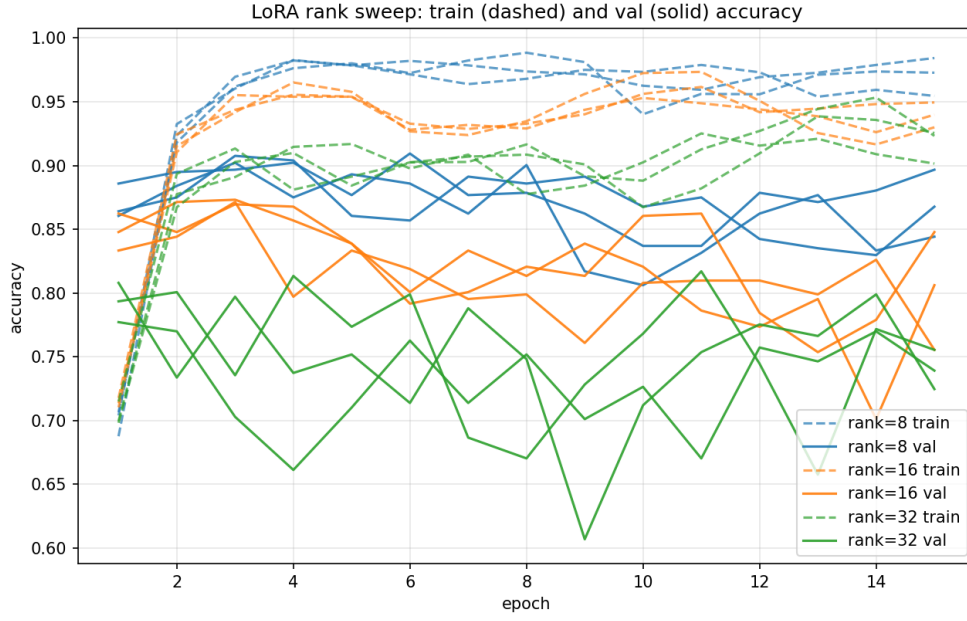


Figure 16: LoRA rank sweep on ResNet50 (conv1, conv2, conv3 in every bottleneck), $\alpha = 2r$, 15 epochs, 3 seeds per rank. Dashed lines are training accuracy and solid lines validation accuracy.

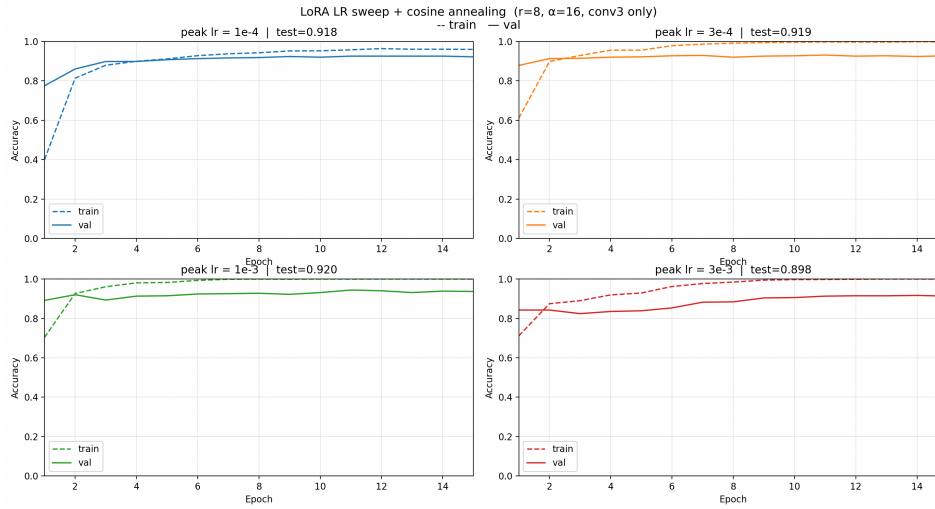


Figure 17: Comparison of accuracy for training, validation and test datasets over 1 run using lora for different configurations of learning rate.

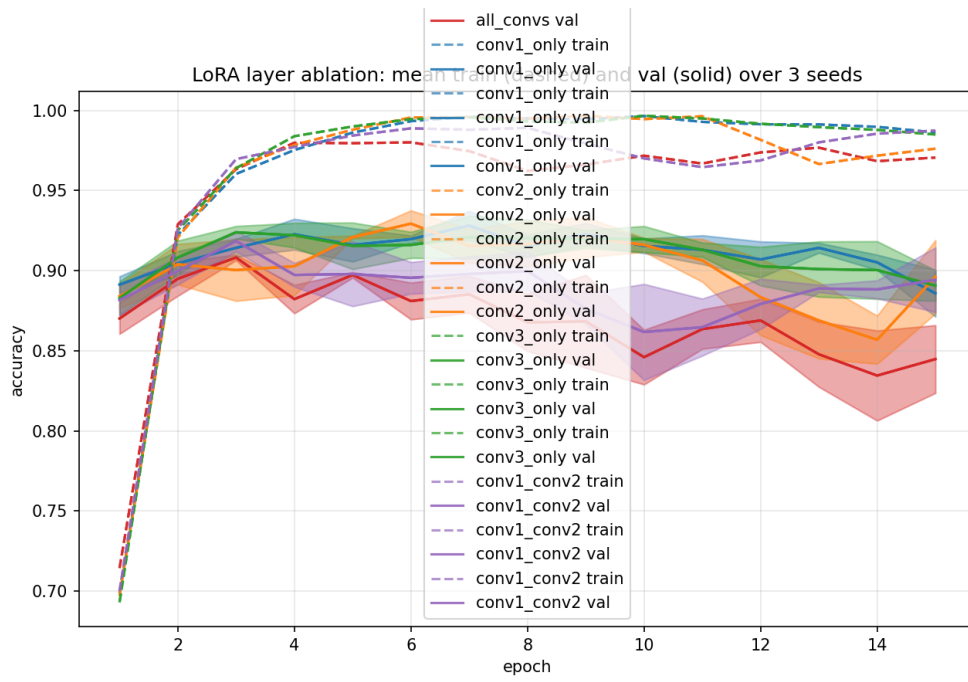


Figure 18: LoRA layer evaluation on how many layers should be injected with the lora layer.